Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
AI and application Development and jira

Practical – 7

Roll No.:T050	Name: Aashu Yadav
Class:TYIT	Batch:1
Date of Assignment:01-8-2026	Date/Time of Submission:01-8-2026

Aim:-
Machine Learning
1. Using any small dataset (e.g., Iris or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.
a. Load dataset using Pandas.

Code:-

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["Target"] = iris.target
print("Dataset:")
print(df.head())
```

Output:-

```
Dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   Target
0       0
1       0
2       0
3       0
4       0
```

b. Perform basic preprocessing (handling missing values or scaling).

Code:-

```
# Check Missing Values
print("\nMissing Values:")
print(df.isnull().sum())
# Handle Missing Values (if any)
imputer = SimpleImputer(strategy="mean")
df[df.columns[:-1]] = imputer.fit_transform(df[df.columns[:-1]])
# Separate Features and Target
X = df.drop("Target", axis=1)
y = df["Target"]
# Feature Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

Output:-

```
Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
Target               0
dtype: int64
```

c. Split dataset into training and testing sets.

Code:-

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
print("\nTraining Samples:", len(X_train))
print("Testing Samples :", len(X_test))
```

Output:-

```
Training Samples: 120
Testing Samples : 30
```

d. Train a simple classifier.

Code:-

```
classifier = DecisionTreeClassifier(random_state=42)
classifier.fit(X_train, y_train)
```

Output:-

```
▾ DecisionTreeClassifier
▸ Parameters
▸ Fitted attributes
```

e. Display training and testing accuracy.

Code:-

```
# Predictions
train_pred = classifier.predict(X_train)
test_pred = classifier.predict(X_test)
# Accuracy
train_accuracy = accuracy_score(y_train, train_pred)
test_accuracy = accuracy_score(y_test, test_pred)
print("\nTraining Accuracy :", round(train_accuracy * 100, 2), "%")
print("Testing Accuracy  :", round(test_accuracy * 100, 2), "%")
```

Output:-

```
Training Accuracy : 100.0 %
Testing Accuracy  : 100.0 %
```

2. Using any small dataset (e.g., your own or a CSV dataset), write a Python program to demonstrate the basic machine learning workflow.

```
# 1. Import Required Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```
✓ 0.0s

a. Load dataset using Pandas.

Code & Output:

```
df = pd.read_csv("titanic.csv")

print("First 5 Records:")
print(df.head())
```
✓ 0.1s

```
First 5 Records:
   PassengerId  Survived  Pclass  \
0          892         0       3
1          893         1       3
2          894         0       2
3          895         0       3
4          896         1       3

                                           Name     Sex   Age  SibSp  Parch  \
0                              Kelly, Mr. James    male  34.5      0      0
1              Wilkes, Mrs. James (Ellen Needs)  female  47.0      1      0
2                     Myles, Mr. Thomas Francis    male  62.0      0      0
3                              Wirz, Mr. Albert    male  27.0      0      0
4  Hirvonen, Mrs. Alexander (Helga E Lindqvist)  female  22.0      1      1

    Ticket     Fare Cabin Embarked
0   330911   7.8292   NaN        Q
1   363272   7.0000   NaN        S
2   240276   9.6875   NaN        Q
3   315154   8.6625   NaN        S
4  3101298  12.2875   NaN        S
```


```
df = pd.read_csv("titanic.csv")

print("First 5 Records:")
print(df.head())
```
✓ 0.1s

```
First 5 Records:
   PassengerId  Survived  Pclass  \
0          892         0       3
1          893         1       3
2          894         0       2
3          895         0       3
4          896         1       3

                                           Name     Sex   Age  SibSp  Parch  \
0                              Kelly, Mr. James    male  34.5      0      0
1              Wilkes, Mrs. James (Ellen Needs)  female  47.0      1      0
2                     Myles, Mr. Thomas Francis    male  62.0      0      0
3                              Wirz, Mr. Albert    male  27.0      0      0
4  Hirvonen, Mrs. Alexander (Helga E Lindqvist)  female  22.0      1      1

    Ticket     Fare Cabin Embarked
0   330911   7.8292   NaN        Q
1   363272   7.0000   NaN        S
2   240276   9.6875   NaN        Q
3   315154   8.6625   NaN        S
4  3101298  12.2875   NaN        S
```


b. Perform basic preprocessing (handling missing values or scaling).

Code & Output:

```
# Check Missing Values
print("\nMissing Values:")
print(df.isnull().sum())
# Drop unnecessary columns
df.drop(["PassengerId", "Name", "Ticket", "Cabin"], axis=1, inplace=True)
# Fill missing values
df["Age"].fillna(df["Age"].mean(), inplace=True)
df["Embarked"].fillna(df["Embarked"].mode()[0], inplace=True)
# Encode categorical columns
le = LabelEncoder()
df["Sex"] = le.fit_transform(df["Sex"])        # male=1, female=0
df["Embarked"] = le.fit_transform(df["Embarked"])
# Separate Features and Target
X = df.drop("Survived", axis=1)
y = df["Survived"]
# Feature Scaling
scaler = StandardScaler()
X = scaler.fit_transform(X)
```
✓ 0.0s

```
Missing Values:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age             86
SibSp            0
Parch            0
Ticket           0
Fare             1
Cabin          327
Embarked         0
dtype: int64
```

c. Split the dataset into training and testing sets.

Code & Output:

```python
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

```
Training Samples: 334
Testing Samples: 84
```

d. Train a simple classifier.

Code & Output:

```python
classifier = DecisionTreeClassifier(random_state=42)
classifier.fit(X_train, y_train)
```

```
DecisionTreeClassifier
▸ Parameters
▸ Fitted attributes
```

e. Display training and testing accuracy.

Code & Output:

```python
# Predictions
train_pred = classifier.predict(X_train)
test_pred = classifier.predict(X_test)
# Accuracy
train_accuracy = accuracy_score(y_train, train_pred)
test_accuracy = accuracy_score(y_test, test_pred)
print("\nTraining Accuracy :", round(train_accuracy * 100, 2), "%")
print("Testing Accuracy  :", round(test_accuracy * 100, 2), "%")
```

```
Training Accuracy : 100.0 %
Testing Accuracy  : 100.0 %
```

1) Comparison of Iris and Titanic Dataset

Iris Dataset	**Titanic Dataset**
Small and clean dataset	Real-world dataset
150 records	891 records
No missing values	Contains missing values
Multi-class classification	Binary classification
Minimal preprocessing	Requires preprocessing and encoding

2) Justification

- **Iris Dataset: Best for learning basic machine learning concepts because it is simple and easy to use.**
- **Titanic Dataset: Better for real-world machine learning as it requires data cleaning, preprocessing, and classification.**

3) Conclusion: The Titanic dataset is more suitable for demonstrating the complete machine learning workflow, while the Iris dataset is ideal for beginners.